

Assignment: Command-Line Hex Viewer

Objective

Build a **command-line hex viewer/editor** that allows users to inspect binary files in a readable hexadecimal + ASCII representation.

You may use **any programming language, libraries, documentation, and AI-assisted development tools** available to you.

The emphasis is on producing a **working, usable, well-designed solution** rather than implementing a large amount of code.

Problem Statement

Develop a command-line application called `hexview` that can open a binary file and display its contents in a traditional hex-editor style.

For example:

```
$ hexview sample.bin
```

could produce:

Offset	Hex	ASCII
00000000	48 65 6C 6C 6F 20 57 6F 72 6C 64 21 00 01 02 03	Hello World!....
00000010	7F 45 4C 46 02 01 01 00 00 00 00 00 00 00 00 00	.ELF.....
00000020	02 00 3E 00 01 00 00 00 40 00 00 00 00 00 00 00	..>.....@.....

The program should work with arbitrary binary files and should **not assume that the input is text**.

Functional Requirements

1. Basic Hex Viewer

The application must accept a binary file as a command-line argument:

```
hexview <file>
```

It must display:

- byte offset
- hexadecimal representation of bytes
- printable ASCII representation
- a consistent number of bytes per row

Non-printable bytes should be represented appropriately in the ASCII section rather than producing terminal control characters.

The offset should be displayed in hexadecimal.

2. File Navigation

Provide a way to inspect a particular portion of a large file without displaying the entire file.

The user should be able to specify at least one of:

```
hexview <file> --offset <offset>
```

or an equivalent interface.

The application should support offsets beyond the beginning of the file and handle invalid offsets gracefully.

3. Search

Implement binary-file searching.

The application should support searching for:

Hexadecimal sequences

For example:

```
hexview sample.bin --search-hex "48 65 6C 6C 6F"
```

Text sequences

For example:

```
hexview sample.bin --search-text "ELF"
```

The output should identify where matches occur.

Multiple occurrences should be handled correctly.

4. Configurable Display

Allow the user to control at least one meaningful aspect of the display, such as:

- number of bytes per row
- number of rows displayed
- starting offset
- ASCII visibility
- uppercase/lowercase hexadecimal representation

For example:

```
hexview sample.bin --width 32
```

Additional Features

Implement **at least three** of the following features.

A. File Information

Provide metadata such as:

- file size
- number of bytes
- file type, where reasonably detectable
- checksum/hash

Example:

```
hexview sample.bin --info
```

B. Interactive Mode

Provide an interactive mode in which the user can navigate through the file without repeatedly restarting the program.

For example:

```
hexview sample.bin --interactive
```

The interface may support commands such as:

```
next
prev
goto 0x1000
search "ELF"
quit
```

The exact command set is your choice.

C. Byte Editing

Allow the user to modify individual bytes.

For example, an interface could support:

```
edit 0x0010 FF
```

The application must ensure that modifications do not accidentally corrupt unrelated bytes.

You should clearly distinguish between viewing a file and modifying a file.

D. Comparison

Allow two binary files to be compared.

For example:

```
hexview --compare file1.bin file2.bin
```

The output should identify meaningful differences between the files.

E. Search Result Navigation

When multiple matches are found, provide a convenient way to inspect the bytes surrounding each match.

For example:

```
Match 1: 0x00000120
Match 2: 0x000004A8
Match 3: 0x00000910
```

The user should be able to inspect a selected occurrence.

F. Multiple Input Formats

Allow search or byte input to accept representations such as:

```
FF 00 A7
```

and

```
0xFF 0x00 0xA7
```

and handle invalid input cleanly.

G. Output Modes

Provide an option to produce machine-readable output, such as:

```
hexview sample.bin --json
```

The exact JSON structure is your design decision.

H. Large-File Handling

The application should remain usable with files significantly larger than available RAM.

Your implementation should avoid unnecessarily loading the complete file into memory.

Command-Line Interface

Design a coherent command-line interface for your application.

At minimum, the following should be possible:

```
hexview <file>
hexview <file> --offset <offset>
hexview <file> --search-hex "<hex sequence>"
hexview <file> --search-text "<text>"
```

The exact syntax may differ if you provide an equally clear interface.

The program should provide useful command-line help:

```
hexview --help
```

Error Handling

The application should handle common failure cases gracefully, including:

- file does not exist
- file cannot be opened
- invalid command-line arguments
- malformed hexadecimal input
- invalid offset
- offset beyond the file
- empty files
- very small files
- very large files

The program should not terminate with an unexplained stack trace for normal user errors.

Testing

Test your application using binary files representing different situations, including:

1. an empty file
2. a very small binary file

3. a file containing printable and non-printable bytes
4. a file containing repeated patterns
5. a relatively large file
6. a file containing arbitrary binary data

You should include a small set of test cases or a testing procedure demonstrating that your implementation works.

Deliverables

Submit:

1. Source Code

Complete source code required to run the application.

2. README

The README should contain:

- how to install/run the application
- command-line usage
- implemented features
- examples
- dependencies
- known limitations

3. Design Notes

Briefly explain:

- your overall architecture
- how binary data is read
- how searching is implemented
- how you handle large files
- which additional features you selected
- any significant design decisions

4. Test Evidence

Provide representative examples demonstrating the major functionality of your application.